GRIT · INTERVIEW HUB • AI PRODUCT ENGINEER · ROUND 1

Interview scheduling, evaluation and badge clearance

Students complete required skills, book one of a limited set of live interview slots, are evaluated by an interviewer, and either earn a badge or a dated retake window. Operations sees who is eligible, booked and unbooked, publishes results, reassigns, handles malpractice — and never loses the history behind a badge. This is a running Next.js 16 + PostgreSQL application, not a mockup.

Next.js 16 · React 19 | PostgreSQL · Prisma 7 | 44 automated tests · 11 suites | Server-enforced RBAC · 3 roles

01 States and what moves them

ONE STATE MACHINE

`NOT_ELIGIBLE` → `ELIGIBLE` → `CONFIRMED` → `SCHEDULED` → `IN_PROGRESS` → `COMPLETED` → `RESULT_PENDING` → `RESULT_PUBLISHED` → **BADGE**

STAGE	STATES	MOVES ON WHEN (OWNER)
Eligibility	`NOT_ELIGIBLE` `ELIGIBLE` `ELIGIBILITY_REVOKED`	Every required skill completes → `ELIGIBLE` (system). Operations can revoke or restore; a revoked student cannot book.
Booking & slot	`CONFIRMED` `CANCELLED` `RESCHEDULED` `NO_SHOW` `COMPLETED`; slot `OPEN` `BOOKED` `CANCELLED` `BLOCKED`	Student books atomically; cancels or reschedules outside 4 hours; operations records no-show; completing an interview closes it.
Interview	`SCHEDULED` `IN_PROGRESS` `COMPLETED` `RESULT_PENDING` `RESULT_PUBLISHED`	Assigned interviewer starts and completes, then submits the evaluation; operations publishes the result.
Result	`PASS` `FAIL` `RETAKE_LATER`	Set on publication and terminal; never edited afterwards.
Badge	`NOT_EARNED` `PENDING_CLEARANCE` `AWARDED` `SUSPENDED` `NULLIFIED`	A pass awards it; a confirmed integrity case suspends or nullifies it while the result record stays.
Retake	`PENDING` `APPROVED` `REJECTED` `SCHEDULED` `COMPLETED`	Requested after `FAIL` / `RETAKE_LATER`; operations decides; approval unlocks booking.
Integrity	`REPORTED` `UNDER_REVIEW` `CONFIRMED` `REJECTED` `CLOSED`	Operations investigates and closes the case with a required badge action and written resolution.

Every transition passes one state machine (**src/lib/state-machine.ts**) plus conditional database updates, so a stale tab or replayed request cannot force an illegal move.

02 Edge cases, assumptions and open questions

WHAT THE PROTOTYPE HANDLES

EDGE CASE	BEHAVIOUR IN THE PROTOTYPE
Two students race for one slot	Row locks plus a conditional update; exactly one wins, the other sees “that slot was just taken” with refreshed availability.
Student already holds a booking	Rejected by a service check and a partial unique index — one active booking per student.
Double-click or retried request	A unique idempotency key returns the original booking instead of creating a second one.
Slot inside the 60-minute lead time	Not offered in availability and rejected again server-side at booking time.
Cancel or reschedule inside 4 hours	Blocked with the policy explained. Outside the window the old booking becomes `RESCHEDULED` and remains in history.
Availability blocked while bookings exist	Confirmed interviews move to the operations reassignment queue; affected students are notified.
Student does not attend	`NO_SHOW` is recorded against the booking; no result is invented.
Evaluation edited or result published twice	State machine rejects both; a submitted evaluation is immutable and a result publishes exactly once.
Fail or retake-later	`retakeEligibleFrom` applies a 14-day (Round 1) or 21-day wait, with attempt caps of 3 and 2; an early review request can be approved.
Malpractice confirmed after a badge	Badge suspended or nullified with a reason and resolution; result history stays and the student is notified.
Eligibility revoked after booking	The booking stays for operations to reassign; the student sees the hold and cannot book again.

ASSUMPTIONS

One timezone (`Asia/Kolkata`), 45-minute slots, one active interview type per journey, operations is the only publisher, in-app notifications only, and no payments. Wait periods and attempt caps are policy constants, not hard-coded UI copy.

OPEN QUESTIONS

Should interviewers see a student’s prior evaluation before the next attempt? Should retakes use the same interviewer cohort for comparability, or a fresh panel to reduce bias? Both are deliberately left as policy decisions, not guessed in code.

44 / 44
automated tests passing across 11 suites

2 × race
concurrent bookings → exactly one wins

3 roles
RBAC enforced in server actions, not the UI

0
console errors across every role and mobile

03 Data model and how double-booking is prevented

DATABASE-LEVEL GUARANTEE

TABLE	KEY FIELDS	PURPOSE
`User` `StudentProfile` `InterviewerProfile`	`email`, `role`; `studentCode`, `eligibilityStatus`, `cohort`; `maxPerDay`	Identity and profiles
`Skill` `StudentSkill` `Badge` `BadgeAward`	`code`, `isRequired`; `status`; `studentId+badgeId`, `awardedAt`	Eligibility and badge clearance
`InterviewerAvailability` `InterviewSlot`	window/date, `slotMinutes`; `startsAt`, `endsAt`, `status`	Availability and bookable slots
`InterviewBooking`	`studentId`, `slotId`, `status`, `idempotencyKey`, `rescheduledFromId`	Never-deleted booking history
`Interview` `InterviewEvaluation` `InterviewResult`	`status`, `outcome`, `retakeEligibleFrom`; three scores, recommendation; `publishedAt`	Session, evaluation, result
`RetakeRequest` `MalpracticeCase`	`status`, `eligibleFrom`; `status`, `badgeAction`, `allegation`, `resolution`	Exception workflows
`Notification` `AuditLog`	`type`, `body`, `link`; actor, action, before/after	Messages and explainability

Booking transaction

one PostgreSQL transaction · no read-then-write race

1 Lock the student and slot

`SELECT … FOR UPDATE` on both rows, in a fixed order to avoid deadlocks.

2 Re-check inside the lock

Eligibility, attempt cap, lead time and slot `OPEN` state are re-read after the lock is held.

3 Flip only if still open

`updateMany({ where: { id, status: "OPEN" } })` must affect exactly `count === 1`; the loser gets `CONFLICT`.

4 Commit booking + interview

Booking, interview row, notification and audit entry commit together or not at all.

DB BACKSTOP

Two partial unique indexes enforce the same rule even if application code regresses: **one `CONFIRMED` booking per student** (`one_active_booking_per_student`) and **one per slot** (`one_active_booking_per_slot`, on `slotId` where `status = 'CONFIRMED'`). A real PostgreSQL integration test races two students with `Promise.allSettled` and asserts one success, one conflict and exactly one booking row.

04 The three most important student messages

EXACT SHIPPED COPY

ELIGIBILITY UNLOCKED when the last required skill completes

> Congratulations! You have completed the required skill milestones and are now eligible for your GRIT interview. Choose an available slot to begin your badge clearance journey.

BOOKING CONFIRMED immediately after the slot is reserved

> Your GRIT interview has been successfully booked. Your interview details are available in your dashboard. Please review your scheduled date and time and be ready before the interview begins.

RESULT PUBLISHED — PASS badge awarded at the same moment

> Congratulations! You have successfully cleared your GRIT interview. Your badge has been awarded and is now available in your dashboard.

The **FAIL** and **RETAKE_LATER** variants keep the same opening and replace the final sentence with the dated retake window. All copy is centralised in **src/lib/notifications.ts** and asserted by tests, so wording cannot drift from the product.

05 The working prototype, including before/after

ALL SCREENS LIVE

Student

- `/student` status, next action, clearance path, result, badge
- `/student/book` real availability + confirmation dialog
- `/student/interviews` and `/student/badge`

Interviewer

- `/interviewer` today's daily schedule
- `/interviewer/availability` set, edit and block windows
- `/interviewer/interviews/[id]` start, evaluate, submit scores

Operations

- `/admin` queues and published outcomes
- booked vs. unbooked students and the "eligible but unbooked" queue
- results, retakes, malpractice, audit and analytics

BEFORE / AFTER

/result-impact renders the same dashboard from real seeded records: **Kabir Shah** before publication, **Meera Krishnan** after a pass and badge award, and **Rohan Verma** after a fail with a dated retake window. Linked from the landing and login screens, so the before/after states are part of the prototype rather than separate slides.

Demo persona sign-in, no password · in-app notifications only.

Live URL supplied after the hosted deploy.